

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

**Duration : 45 Minutes
Total Marks:20**

Annexure A **INTEGRATED**

EXPERIMENTS

Code of Conduct:

I J O Joanna Divine(192572003) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Joanna

EXPERIMENT 1: DECISION TREE CLASSIFICATION

1. Objective

To implement and evaluate a Decision Tree Classifier using the Iris dataset. The dataset is pre-processed, divided into training and testing sets, and classified using the Gini Index as the splitting criterion.

2. Software Requirements

- Python 3.x
- NumPy
- Pandas
- Scikit-learn
- Matplotlib

Install libraries if required:

```
pip install numpy pandas scikit-learn matplotlib
```

3. Procedure

1. Load the Iris dataset.
2. Convert the dataset into a Pandas Data Frame.
3. Check for missing values and data types.
4. Handle missing numerical values using the mean.
5. Encode the target class.
6. Split the dataset into training and testing data frame

7. *Build a Decision Tree using the Gini Index.*
 8. *Display the tree structure.*
 9. *Identify the root node.*
 10. *Calculate accuracy, confusion matrix and classification report.*
 11. *Display at least two misclassified instances.*
-

4. Python Program

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import Decision Tree Classifier, export_text

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset

iris = load_iris()
```

```
X = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
y = pd.Series(iris.target, name="target")
```

```
# Display first 5 rows and data types
```

```
print("First 5 Rows:")
```

```
print(X.head())
```

```
print("\nData Types:")
```

```
print(X.dtypes)
```

```
# Handle missing values
```

```
X = X.fillna(X.mean())
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.30, random_state=42, stratify=y  
)
```

```
# Create Decision Tree using Gini Index
```

```
model = DecisionTreeClassifier(  
    criterion="gini",  
    random_state=42  
)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Display tree
```

```
print("\nDecision Tree:")
```

```
print(export_text(model, feature_names=list(X.columns)))
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
# Confusion Matrix
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```
print(classification_report(
```

```
y_test, y_pred, target_names=iris.target_names
))
```

```
# Misclassified instances
```

```
print("\nMisclassified Instances:")
```

```
count = 0
```

```
for i in range(len(y_test)):
```

```
    if y_test.iloc[i] != y_pred[i]:
```

```
        print(
```

```
            "Actual:", iris.target_names[y_test.iloc[i]],
```

```
            "Predicted:", iris.target_names[y_pred[i]],
```

```
            "Features:", X_test.iloc[i].values
```

```
        )
```

```
        count += 1
```

```
if count == 2:
```

```
    break
```

```
# Root node
```

```
root = model.tree_.feature[0]
```

```
print("\nRoot Node:", X.columns[root])
```

```
print("Root Node is selected because it gives the best Gini-based split.")
```

5. Sample Output

First 5 Rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Accuracy: 0.9333

Confusion Matrix:

[[15 0 0]

[0 12 3]

[0 0 15]]

Root Node

Petal length (cm) <= 2.45

The petal length is selected as the root node because it provides the best separation of the classes according to the Gini Index.

Example Misclassified Instances

Actual: versicolor

Predicted: virginica

Features: [6.3 3.3 4.7 1.6]

Actual: versicolor

Predicted: virginica

Features: [6.0 3.4 4.5 1.6]

The exact output can vary if the train-test split or random state is changed.

6. Performance Analysis

The Decision Tree achieves approximately 93.33% accuracy on the test data.

The confusion matrix shows that:

- Most Setosa samples are correctly classified.*
- Most Versicolor samples are correctly classified.*
- Most Virginica samples are correctly classified.*
- A few Versicolor samples are incorrectly classified as Virginica.*

Therefore, the model provides good classification performance on the Iris dataset.

7. Result

The Decision Tree Classifier was successfully implemented using the Gini Index. The root node was identified as Petal Length, and the classifier achieved approximately 93.33% test accuracy.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

1. Objective

To implement the Q-Learning algorithm for a simple 4×4 Grid World and observe how the agent learns an optimal policy through repeated interaction with the environment.

2. Environment

The environment contains:

- Grid size: 4×4*

- *Start state: 0*
- *Goal state: 15*
- *Obstacles: 5 and 10*
- *Actions:*
 - *Up*
 - *Down*
 - *Left*
 - *Right*

Reward Structure

<i>Situation</i>	<i>Reward</i>
<i>Reach Goal</i>	<i>+10</i>
<i>Normal Step</i>	<i>-1</i>
<i>Hit Obstacle/Invalid Move</i>	<i>-5</i>

Grid

```
+----+----+----+----+
| S |   |   |   |
+----+----+----+----+
|   | X |   |   |
+----+----+----+----+
|   |   | X |   |
+----+----+----+----+
|   |   |   | G |
+----+----+----+----+
```

S = Start

X = Obstacle

G = Goal

3. Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$$

Where:

- α = Learning rate
- γ = Discount factor
- r = Reward
- s = Current state
- a = Current action
- s' = Next state

Hyperparameters

Learning rate (α) = 0.1

Discount factor (γ) = 0.9

Exploration rate (ϵ) = 0.2

Episodes = 500

4. Python Program

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
# Grid settings
```

```
N = 4
```

```
states = 16
```

```
actions = 4
```

```
# Actions: Up, Down, Left, Right
```

```
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
names = ["Up", "Down", "Left", "Right"]
```

start = 0

goal = 15

obstacles = {5, 10}

Hyperparameters

alpha = 0.1

gamma = 0.9

epsilon = 0.2

Q-table

Q = np.zeros((states, actions))

def step(state, action):

row, col = divmod(state, N)

dr, dc = moves[action]

nr = row + dr

nc = col + dc

Invalid movement

if nr < 0 or nr >= N or nc < 0 or nc >= N:

return state, -5, False

*new_state = nr * N + nc*

Obstacle

if new_state in obstacles:

return state, -5, False

Goal

```
if new_state == goal:  
    return new_state, 10, True
```

```
return new_state, -1, False
```

```
rewards = []
```

```
# Training
```

```
for episode in range(1, 501):
```

```
    state = start
```

```
    total_reward = 0
```

```
    for step_no in range(100):
```

```
# Epsilon-greedy action selection
if random.random() < epsilon:
    action = random.randint(0, 3)
else:
    action = np.argmax(Q[state])

next_state, reward, done = step(state, action)

# Q-learning update
Q[state, action] = Q[state, action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)

total_reward += reward
```

```
state = next_state
```

```
if done:
```

```
    break
```

```
rewards.append(total_reward)
```

```
# Display Q-values
```

```
if episode in [1, 50, 100]:
```

```
    print("\nEpisode", episode)
```

```
    print("State 0:", np.round(Q[0], 2))
```

```
    print("State 6:", np.round(Q[6], 2))
```

```
# Display final Q-table
```

```
print("\nFinal Q-Table:")
```

```
print(np.round(Q, 2))
```

```
# Display learned policy
```

```
print("\nFinal Learned Policy:")
```

```
for state in range(states):
```

```
    if state == goal:
```

```
        print("G", end=" ")
```

```
    elif state in obstacles:
```

```
        print("X", end=" ")
```

```
    else:
```

```
        best_action = np.argmax(Q[state])
```

```
        print(names[best_action][0], end=" ")
```

```
if (state + 1) % N == 0:
```

```
    print()
```

```
# Plot cumulative reward
```

```
plt.plot(rewards)
```

```
plt.xlabel("Episode")
```

```
plt.ylabel("Cumulative Reward")
```

```
plt.title("Q-Learning: Reward per Episode")
```

```
plt.grid()
```

```
plt.show()
```

5. Sample Output

Episode 1

State 0: [-0.5 -0.1 -0.5 0.0]

State 6: [0. 0. 0. 0.]

Episode 50

State 0: [-3.87 -2.28 -4.27 0.82]

State 6: [-0.55 -0.50 -0.50 1.97]

Episode 100

State 0: ...

State 6: ...

Final Q-Table:

[[-3.87 -2.28 -4.27 0.82]

...

...]

Final Learned Policy:

R R D D

D X D D

D D X D

R R R G

U = Up, D = Down, L = Left, R = Right.

The exact Q-values may vary because Q-Learning uses random exploration.

6. Learned Policy

After training, the agent selects the action having the highest Q-value for each state.

For example:

R → Move Right

D → Move Down

X → Obstacle

G → Goal

A possible optimal path is:

Start

↓

↓

↓

→

→

→

Goal

The learned policy attempts to reach the goal while avoiding obstacles and unnecessary movements.

7. Convergence Behaviour

Initially, the Q-table contains only zeros. Therefore, the agent does not know which action is useful.

During training:

- 1. The agent explores different actions.*
- 2. It receives rewards and penalties.*
- 3. Q-values are updated.*
- 4. Actions leading toward the goal obtain higher Q-values.*
- 5. Actions leading to obstacles obtain lower Q-values.*
- 6. The cumulative reward generally improves as learning progresses.*

Therefore, the Q-values gradually stabilize and the agent learns a better policy.

8. Result

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World. The agent learned Q-values through repeated interaction with the environment and obtained a policy that attempts to reach the goal while avoiding obstacles.

FINAL CONCLUSION

In Experiment 1, a Decision Tree Classifier was implemented on the Iris dataset. The model achieved approximately 93.33% accuracy, demonstrating good classification performance.

In Experiment 2, Q-Learning was implemented in a 4×4 Grid World. The agent learned from rewards and penalties and gradually developed a policy for reaching the goal while avoiding obstacles.

Thus, the experiments demonstrate two important Machine Learning approaches:

- Decision Tree: Supervised Learning*
- Q-Learning: Reinforcement Learning*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 –	Level 3 –	Level 2 –	Level 1 –
----------	-----------	-----------	-----------	-----------	-----------

		Exemplary	Proficient	Developing	Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation